Traffic-Knowledge-Augmented Path Planning for Autonomous Vehicles in Internet of Things

Danni Chen, Chao Lu, Shijie Wu, Xuemei Chen, and Jianwei Gong, *Member, IEEE*

***Abstract*—Traffic knowledge is essential for safe, efficient, and regulation-compliant autonomous driving. While existing path planning methods often encode only partial traffic rules or focus on specific scenarios, they lack adaptability to diverse and dynamic environments. The Internet of Things (IoT) enables vehicles to access rich, real-time traffic knowledge, yet most IoT-based approaches emphasize long-distance route optimization and overlook local path planning. This article proposes a traffic knowledge-augmented path planning model (KARS) that integrates a knowledge graph representation of traffic knowledge with a reachable set (RS)-based planning framework in IoT environments. The knowledge graph systematically encodes spatial, temporal, and speed constraints, while the RS method enables the planner to dynamically adapt to real-time traffic knowledge. KARS is evaluated in multiconstraint simulation environments with static, semidynamic, and dynamic traffic knowledge settings. Compared to representative baselines, it improves driving safety by increasing traffic success rates in complex environments, raises maximum driving speed to enhance operational efficiency, and produces smoother acceleration profiles for improved ride comfort. These results validate the effectiveness of combining traffic knowledge with RS planning for safe, efficient, and adaptable autonomous driving in IoT-enabled urban scenarios.**

***Index Terms*—Autonomous vehicles, Internet of Things (IoT), reachable sets (RSs), traffic knowledge.**

I. Introduction

TRAFFIC knowledge plays a vital role in ensuring driving safety and maintaining traffic order. To build a highly reliable autonomous driving system, path planning algorithms for autonomous vehicles must not only consider the avoidance of environmental obstacles and the motion constraints of the vehicle itself, but also incorporate and make full use of traffic knowledge.

At present, some studies have begun to integrate certain aspects of traffic knowledge into the path planning process. For example, a navigation algorithm for autonomous vehicles [1] has been proposed to incorporate traffic constraints such as collision avoidance and intersection stopping; a hierarchical decision-making and planning model [2] has been developed, which combines static traffic regulation information with dynamic traffic participant information to improve efficiency and safety in lane-change scenarios; and a spatio-temporal risk field model [3] has been introduced, which considers traffic elements such as solid and dashed lane markings to assess traffic risks and plan trajectories for intelligent connected vehicles on nonintersection road segments. However, these methods are mostly designed for specific scenarios, consider only partial rules from traffic knowledge, and lack adaptability to diverse traffic environments.

The rapid development of the Internet of Things (IoT) provides the conditions for path planning that fully accounts for traffic knowledge, as it can offer autonomous vehicles comprehensive and rich sources of dynamic road information [4]. In an IoT environment, autonomous vehicles can obtain more complete and real-time traffic knowledge by communicating with road infrastructure and other vehicles [5]. Some studies have already applied IoT to autonomous vehicle planning [6]. For instance, a routing algorithm based on vehicle motion distribution has been proposed [4] to obtain predicted trajectories and transmission decisions in vehicular networks; and a personalized route planning algorithm has been developed [7] that considers spatio-temporal constraints collected by IoT. However, existing methods mostly focus on long-distance, map-level route planning, while neglecting local path planning during individual vehicle operation, thus failing to fully exploit the potential of IoT in fine-grained vehicle path planning. Therefore, there is an urgent need to develop a planning method that can systematically integrate traffic knowledge, flexibly adapt to road changes, and incorporate traffic knowledge into real-world scenarios. To address this issue, this article proposes a path planning method for complex urban environments, which systematically integrates traffic knowledge and adapts dynamically to varying road constraints while complying with traffic regulations, thereby generating safe, compliant, and efficient driving paths. The main contributions of this article are as follows.

1) A traffic knowledge-augmented path planning framework is proposed. By systematically considering traffic

Received 15 August 2025; accepted 21 September 2025. Date of publication 24 September 2025; date of current version 20 November 2025. This work was supported in part by the National Science and Technology Major Project under Grant 2022ZD0115503, in part by Beijing Natural Science Foundation under Grant L247033, and in part by the National Natural Science Foundation of China under Grant 52372405. *(Corresponding authors: Chao Lu; Xuemei Chen.)*

Danni Chen was with the School of Mechanical Engineering, Beijing Institute of Technology, Beijing 100081, China. She is now with the Department of Aeronautical and Aviation Engineering, The Hong Kong Polytechnic University, Hong Kong, SAR, China (e-mail: danni.chen@connect.polyu.hk).

Chao Lu and Shijie Wu are with the School of Mechanical Engineering, Beijing Institute of Technology, Beijing 100081, China (e-mail: chaolu@bit.edu.cn; wushijie1017@bit.edu.cn).

Xuemei Chen is with the School of Mechanical Engineering, Beijing Institute of Technology, Beijing 100081, China, and also with the Advanced Technology Research Institute, Beijing Institute of Technology, Jinan, Shandong 250011, China (e-mail: chenxue781@126.com).

Jianwei Gong is with the School of Mechanical Engineering, Beijing Institute of Technology, Beijing 100081, China, and also with Zhengzhou Research Institute, Beijing Institute of Technology, Zhengzhou 450000, China (e-mail: gongjianwei@bit.edu.cn).

Digital Object Identifier 10.1109/JIOT.2025.3613790

2327-4662 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Indian Institute Of Technology Jammu. Downloaded on August 19,2026 at 16:10:56 UTC from IEEE Xplore. Restrictions apply.

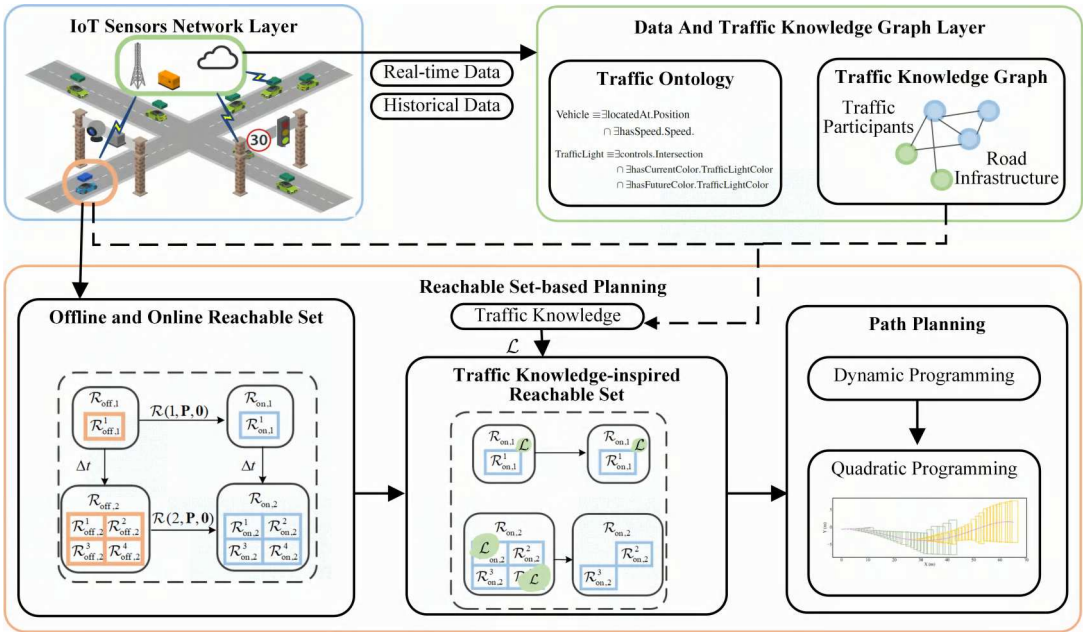


Fig. 1. Framework of the proposed KARS model.

rules, environmental obstacles, and behavioral constraints, it makes up for the insufficient utilization of traffic knowledge in existing methods and improves safety.

2) Using real-time traffic data from the IoT, this approach focuses on the local path planning level rather than the route level, enhancing the real-time adaptability of vehicle path planning and improving driving efficiency and comfort.

II. Traffic Knowledge-Augmented Representation in IoT

A. Overall Architecture in IoT

IoT provides an ego vehicle with rich traffic information, which serves as a crucial foundation for enhancing vehicle planning. This traffic information should encompass complex and detailed information about road infrastructure, such as lanes, sidewalks, traffic signs, crosswalks, and traffic lights, and also should include information about surrounding vehicles in an IoT environment.

Knowledge graphs as an intermediary between IoT and applications can uniformly manage all IoT devices [8]. Traffic representations based on knowledge graphs offer a rich structure and semantics by incorporating diverse types of knowledge provided by IoT. Suggested by [9] and [10], our study builds a structure of overall architecture based on IoT and knowledge graphs, as shown in Fig. 1.

1) IoT Sensors Network Layer: This layer is divided into two stages: the road stage and the cloud stage. The road stage includes data collection and communication modules on both the vehicle side and the roadside. On the vehicle side, connected vehicles are equipped with sensors and communication devices that collect real-time information, such as speed and lane position. The roadside infrastructure includes sensors like cameras, radar, and roadside units (RSUs). Together, these devices create a collaborative sensing network that facilitates communication between vehicles and infrastructure.

The cloud stage consists of mobile edge computing (MEC) nodes and remote cloud servers. These components work together to process data from the road layer. Connected vehicles communicate through the PC5 interface with other vehicles, pedestrians, and infrastructure, enabling scenarios, like V2V and V2I. This ensures low-latency, highly reliable services. Historical data collected from vehicles and roadside sensors is transmitted to the cloud layer, where it is used to build a high-precision traffic map based on knowledge graphs. The cloud continuously updates and refines the knowledge graph with real-time information and sends the updated map back to RSUs, which then transmit it to connected vehicles via V2I communication.

2) Data and Knowledge Graph Layer: This layer is responsible for collecting, processing, and storing relevant IoT data. In general, some road infrastructure does not have hard real-time requirements and can rely on IoT historical data. For real-time changes, such as the sudden appearance of an accident zone, information needs to be obtained through IoT-based real-time communication. A knowledge ontology and knowledge graphs related to road traffic are developed. The constructed ontology includes entities such as vehicles, road infrastructure, and other IoT sensor data. By assigning real instance values of roads to the ontology based on the collected data, we create a knowledge graph representing the real roads. When dynamic information is transmitted from the IoT, the corresponding graph is modified, updating the map representation.

3) RS-Based Planning: This layer addresses path planning for the ego vehicle based on reachable sets (RSs). Each connected vehicle is considered a smart object. On connected vehicles, the knowledge graph provided by the cloud is used to plan driving trajectories. The planning algorithm takes into account dynamic information about nearby traffic participants

Authorized licensed use limited to: Indian Institute Of Technology Jammu. Downloaded on August 19,2026 at 16:10:56 UTC from IEEE Xplore. Restrictions apply.

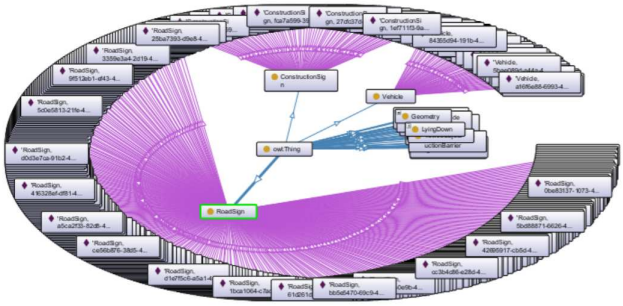


Fig. 2. Illustration of traffic knowledge graph.

and road infrastructure by querying the knowledge graph. Vehicles can adjust their trajectories in real time based on updates from the knowledge graph, ensuring optimal paths that are both safe and efficient.

B. Knowledge Ontology Construction and Knowledge Graph Extraction

To account for traffic knowledge, urban traffic maps need to represent complex and detailed road topology, including lanes, sidewalks, traffic signs, and traffic lights. Accurate representation requires a high-level semantics of a driving scene and relevant context information.

Knowledge graph-based traffic representation offers rich structural and semantic information [10]. Initially introduced by Google in 2012, knowledge graphs serve as a form of knowledge base, describing concepts and relationships in the objective world. They excel in knowledge storage, querying, and reasoning, with their rules being defined in a syntax that is readable by both humans and machines. Currently, there are some studies focused on constructing knowledge graphs for urban traffic and developing applications based on these graphs [11], [12], [13], [14]. These studies demonstrate that knowledge graphs have great potential in representing complex urban traffic environments and handling traffic regulations, providing strong support for map presentation and path planning in autonomous driving systems.

The knowledge ontology is a general representation of the entities and their relationships within the knowledge graph. An ontology serves as a summary and abstraction of knowledge, containing formal definitions of entities and their relationships. An ontology is a broad data model that does not specifically describe individual information.

Suggested by [13], the modeling of ontology is implemented. We consider traffic participants and road infrastructure, defining the ontology into the following main categories.

For example, in Fig. 2, the ontology is composed of triples, and the extraction of entity categories is based on data. At present, for knowledge graph modeling of urban geographic environments, the widely adopted foundational definition is GeoSPARQL, upon which triples are built. For lanes and traffic signs, our process is as follows: a lane belongs to a region, and a traffic sign belongs to a region. Once these two triples are constructed, reasoning over the knowledge graph allows us to infer that the traffic sign is located on the lane.

1) *Traffic Participants:* Traffic participants can include various subclasses, such as bicycles, pedestrians, and vehicles. This article focuses solely on vehicles, recording their position and speed information

$$\text{Vehicle} \equiv \exists \text{locatedAt.Position} \cap \exists \text{hasSpeed.Speed}. \tag{1}$$

2) *Road Infrastructure:* Road infrastructure refers to the facilities on the road that impact vehicle movement. Lanes need to connect to the lanes in front, behind, and to the left and right. Lane slices need to be associated with lane width, road boundary positions, and lane line attributes. Other map elements are shown below. To model the precise positions, shapes, and sizes of all map elements, "hasShape" relations are introduced for each. Each shape is represented with a subclass of the GeoSPARQL Simple Features ontology concept sf:Geometry. To create a connection between traffic participants and road infrastructure, the "isOn" relation is introduced

$$\begin{aligned}\text{Lane} \equiv\ & \exists \text{hasNextLane.Lane} \cup \exists \text{hasPreviousLane.Lane} \\ & \cup \exists \text{hasLeftLane.Lane} \cup \exists \text{hasRightLane.Lane}\end{aligned} \tag{2}$$

$$\begin{aligned}\text{LaneSlice} \equiv\ & \exists \text{laneHasSlice}^{-1}.\text{Lane} \cap \exists \text{laneSliceHasWidth}.\mathbb{R} \\ & \cap \exists \text{hasLaneMarking.LaneMarking}\end{aligned} \tag{3}$$

$$\begin{aligned}\text{StopArea} \equiv\ & \text{TrafficLightStopArea} \cup \text{YieldStopArea} \cup \text{StopSignArea}\end{aligned} \tag{4}$$

$$\text{Walkway} \equiv \exists \text{walkwayIsNextTo.Lane} \tag{5}$$

$$\text{PedCrossing} \equiv \leq 2 \text{connectsWalkways.Walkway} \tag{6}$$

$$\text{Intersection} \equiv \exists \text{hasLane.Lane} \cap \exists \text{controlledBy.TrafficLight} \tag{7}$$

$$\begin{aligned}\text{TrafficLight} \equiv\ & \exists \text{controls.Intersection} \cap \exists \text{hasCurrentColor.TrafficLightColor} \\ & \cap \exists \text{hasFutureColor.TrafficLightColor}\end{aligned} \tag{8}$$

$$\text{RoadBlock} \equiv \cap \exists \text{hasType.} \{\text{Construction, Accident}\} \tag{9}$$

$$\begin{aligned}\text{SpeedLimitZone} \equiv\ & \exists \text{hasSpeedLimit.SpeedLimit} \cap \exists \text{hasSpeedLimitValue.XSD:integer}.\end{aligned} \tag{10}$$

Our knowledge graph is established with reference to the literature [15]. The graph is developed using Web Ontology Language (OWL), which supports the formal representation of traffic-related objects and events. Our design distinguishes between sequential scenes (representing spatiotemporal continuity) and frame scenes (instantaneous states). Entities are categorized into static participants (e.g., lanes and traffic signs)

Authorized licensed use limited to: Indian Institute Of Technology Jammu. Downloaded on August 19,2026 at 16:10:56 UTC from IEEE Xplore. Restrictions apply.

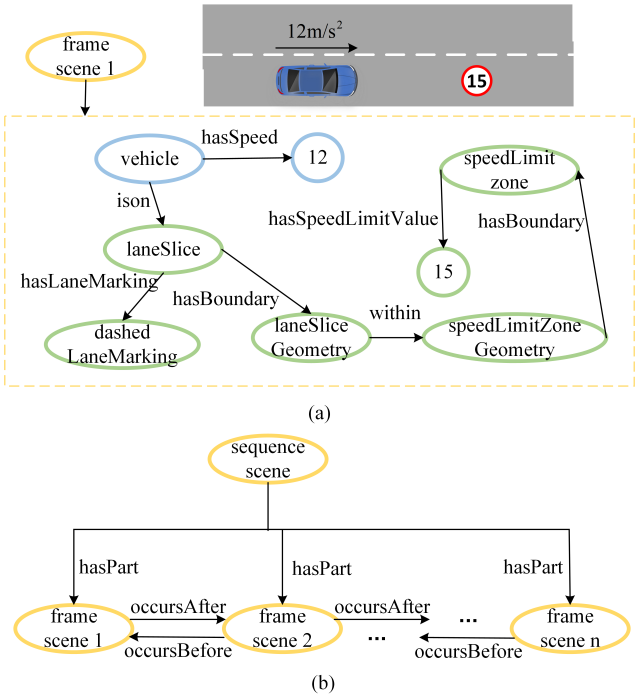


Fig. 3. Knowledge graph of an example scenario. (a) Scene illustrates a speed limit zone and a lane slice. (b) Multiple scenes constitute a sequence of scenes as a knowledge graph.

and dynamic participants (e.g., vehicles). Static participants, such as lanes and traffic signs, are treated as historical data and remain consistent in sequential scenes. Real-time data, such as vehicle states and traffic light status, as dynamic participants, vary by scene and are updated at the frame level. This design enables the fusion of real-time observations with static road knowledge.

After establishing the ontology, instance knowledge was added to create a complete knowledge graph. Suggested by [13], the modeling of road scenarios and the extraction of the knowledge graphs based on ontology are implemented. For example, in Fig. 3, by importing data into the ontology, the knowledge graph for the current scene can be modeled. At this point, lane lines and speed limits should remain unchanged and can rely on historical data, while the host vehicle's speed is updated for each frame as real-time data, forming a continuous sequence of scenes.

Using the SPARQL query language, relevant information can be efficiently searched within the knowledge graph. By querying, the ego vehicle can obtain traffic knowledge regarding surrounding vehicles and road infrastructure. Surrounding vehicles and road infrastructure information collectively form the traffic knowledge set $\mathcal{L}$, which includes speed, time, and spatial limitations. The pseudocode below in Algorithm 1 is a demonstration of querying the current speed limit on the road.

III. RS Generation Considering Traffic Knowledge

The core idea of RS-based algorithms is to calculate all possible reachable states from the current state of a vehicle and then select the optimal path within these RSs to achieve safe and efficient planning. This approach is particularly suitable for planning tasks in complex environments [16], [17], [18].

Algorithm 1 Query Speed Limit Information for Lane Slice

1: **Execute SPARQL Query:**
2: RESULT ← EXECUTE SPARQL QUERY
getSpeedLimitValue(speedLimitZone)
SELECT laneSlice, speedLimitValue
WHERE
laneSlice IS A LaneSlice
laneSlice HAS BOUNDARY laneSliceGeometry
speedLimitZone HAS BOUNDARY
speedLimitZoneGeometry
FILTER(roadSliceGeometry WITHIN
FILTER(speedLimitZoneGeometry)
speedLimitZone HAS speedLimitValue limitValue
3: **Process Query Results:**
4: $\mathcal{L}$ ← RESULT.limitValue

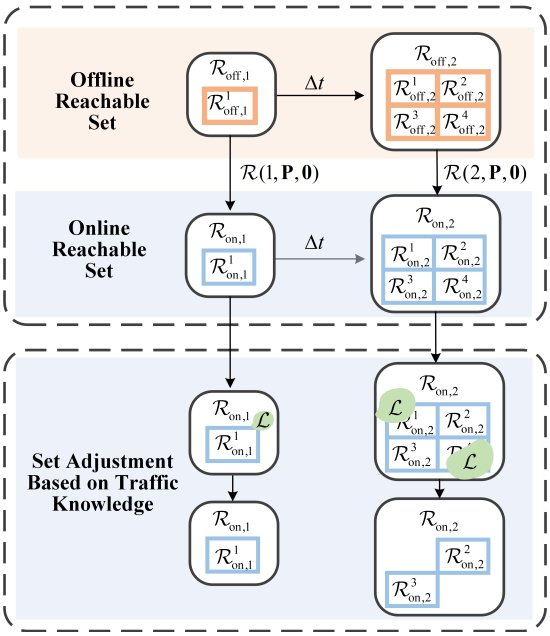


Fig. 4. Illustration of an RS generation considering traffic knowledge.

The computation of the RS is divided into offline reachable stage, online reachable stage, and online adjustment stage, as shown in Fig. 4. In the offline stage, the RS is determined based solely on vehicle dynamics, without considering the initial state of a vehicle and traffic knowledge. In the online stage, the initial state of a vehicle is taken into consideration. In the online adjustment stage, traffic knowledge is queried from the knowledge graph. This knowledge is then applied to adjust the online reachable area, projecting it onto the real road.

A. Offline and Online RS Generation

In this study, the dynamics of the main vehicle are abstractly modeled using a point-mass model. This abstraction ensures that the RS of the high-fidelity model is always a subset of that

Authorized licensed use limited to: Indian Institute Of Technology Jammu. Downloaded on August 19,2026 at 16:10:56 UTC from IEEE Xplore. Restrictions apply.

of the simplified mode. By imposing limits on acceleration and speed, the dynamic boundaries of the vehicle are maintained

$$\begin{pmatrix} p_x(k+1) \\ \dot{p}_x(k+1) \\ p_y(k+1) \\ \dot{p}_y(k+1) \end{pmatrix} = \begin{pmatrix} 1 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} p_x(k) \\ \dot{p}_x(k) \\ p_y(k) \\ \dot{p}_y(k) \end{pmatrix} + \begin{pmatrix} \frac{1}{2}\Delta t^2 & 0 \\ \Delta t & 0 \\ 0 & \frac{1}{2}\Delta t^2 \\ 0 & \Delta t \end{pmatrix} \begin{pmatrix} a_x(k) \\ a_y(k) \end{pmatrix} \tag{11}$$

$$\underline{\dot{p}}_x \le \dot{p}_x(k) \le \overline{\dot{p}}_x, \quad \underline{\dot{p}}_y \le \dot{p}_y(k) \le \overline{\dot{p}}_y \tag{12}$$

$$\underline{a}_x \le a_x(k) \le \overline{a}_x, \quad \underline{a}_y(k) \le a_y \le \overline{a}_y \tag{13}$$

where x represents the longitudinal direction along a road and y signifies the lateral direction; p represents the position of a vehicle; $\dot{p}$ represents the speed of a vehicle; and a represents the acceleration of a vehicle. k represents the time step with an interval of Δt. An overline indicates the maximum value, while an underline indicates the minimum value.

$\mathcal{R}(k, p, a)$ is used to represent the possible state the vehicle can reach at step k, where the vehicle's initial position is p and the input acceleration is a. $\mathcal{R}(k, \mathbf{P}, \mathbf{A})$ represents all reachable states of the vehicle at step k, namely the RS, where $\mathbf{P}$ denotes the set of initial positions and $\mathbf{A}$ denotes the interval of acceleration

$$\mathcal{R}(k, \mathbf{P}, \mathbf{A}) := \{\mathcal{R}(k, p, a) \mid p \in \mathbf{P} \quad \forall \tau \in [t_0, t_0 + k\Delta t] : a(\tau) \in \mathbf{A}\}. \tag{14}$$

As referenced in [18], the computation of $\mathcal{R}(k, p, \mathbf{A})$ can be split into two components. For derivation, let us introduce the Minkowski sum of a singleton a and a set B as $a \oplus B := \{a + b \mid b \in B\}$.

Since the system exhibits a linear dynamic structure and satisfies the superposition principle, the computation of the online RS $\mathcal{R}(k, \mathbf{P}, \mathbf{A})$ can be further decomposed into the following two components.

1) *Offline RS* $\mathcal{R}(k, \mathbf{0}, \mathbf{A})$*:* Generated exclusively from the acceleration control space, without considering the vehicle's initial state.
2) *Initial State* $\mathcal{R}(k, p, \mathbf{0})$*:* Determined by the evolution of the initial state under zero control input.

$$\begin{aligned} \mathcal{R}(k, p, \mathbf{A}) &= \begin{pmatrix} p_x(k) \\ \dot{p}_x(k) \\ p_y(k) \\ \dot{p}_y(k) \end{pmatrix} \\ &= \begin{pmatrix} 1 & k\Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & k\Delta t \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} p_x(0) \\ \dot{p}_x(0) \\ p_y(0) \\ \dot{p}_y(0) \end{pmatrix} \\ &\oplus \begin{pmatrix} \frac{1}{2}(k\Delta t)^2 & 0 \\ (k\Delta t) & 0 \\ 0 & \frac{1}{2}(k\Delta t)^2 \\ 0 & (k\Delta t) \end{pmatrix} \end{aligned}$$

$$\begin{aligned} &= \begin{pmatrix} 1 & k\Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & k\Delta t \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} p_x(0) \\ \dot{p}_x(0) \\ p_y(0) \\ \dot{p}_y(0) \end{pmatrix} \\ &\oplus \begin{pmatrix} \frac{1}{2}(k\Delta t)^2 & 0 \\ (k\Delta t) & 0 \\ 0 & \frac{1}{2}(k\Delta t)^2 \\ 0 & (k\Delta t) \end{pmatrix} \begin{pmatrix} \mathbf{0} \\ \mathbf{0} \end{pmatrix} \\ &+ \begin{pmatrix} 1 & k\Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & k\Delta t \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \mathbf{0} \\ \mathbf{0} \\ \mathbf{0} \\ \mathbf{0} \end{pmatrix} \\ &\oplus \begin{pmatrix} \frac{1}{2}(k\Delta t)^2 & 0 \\ (k\Delta t) & 0 \\ 0 & \frac{1}{2}(k\Delta t)^2 \\ 0 & (k\Delta t) \end{pmatrix} \\ &= \mathcal{R}(k, p, \mathbf{0}) \oplus \mathcal{R}(k, \mathbf{0}, \mathbf{A}). \end{aligned} \tag{15}$$

The offline RS can be computed and stored in advance with an initial position of $\mathbf{0}$ and an acceleration interval of $\mathbf{A}$. During the offline stage, the offline RS is calculated for each step through forward propagation. Define the query functions for obtaining position, velocity, and time step from the RS $\mathcal{R}$ as $Q_p(R)$, $Q_v(R)$, and $Q_k(R)$, respectively,

$$\mathbf{P_k} := Q_p(\mathcal{R}(k, \mathbf{0}, \mathbf{A})) \tag{16}$$

$$\mathcal{R}(k+1, \mathbf{0}, \mathbf{A}) = \mathcal{R}(1, \mathbf{P_k}, \mathbf{A}). \tag{17}$$

For convenience of representation, the offline RS $\mathcal{R}(k, \mathbf{0}, \mathbf{A})$ at time step k, is abbreviated as $\mathcal{R}_{\text{off},k}$. The online RS $\mathcal{R}(k, \mathbf{P}, \mathbf{A})$ at time step k is abbreviated as $\mathcal{R}_{\text{on},k}$.

To facilitate calculations, the RSs are discretized. First, the offline RS is discretized.

1) At each time step k, the offline RS $\mathcal{R}_{\text{off},k}$ is partitioned into sub-RSs $\mathcal{R}^i_{\text{off},k}$ using grid division.
2) If $\mathcal{R}^i_{\text{off},k}$ can reach $\mathcal{R}^j_{\text{off},k+1}$, the transition relation matrix between sub-RSs is set as $T(j, i) = 1$; otherwise, it is set to 0. Let $\mathcal{P}^i_k$ denotes the state set of the RS $\mathcal{R}^i_{\text{off},k}$ at step k

$$\mathcal{P}^i_k := Q_p\left(\mathcal{R}^i_{\text{off},k}\right) \tag{18}$$

$$T(j, i) = \begin{cases} 1, & \text{if } \mathcal{R}\left(1, \mathcal{P}^i_k, A\right) \cap \mathcal{R}^j_{\text{off},k+1} \neq \emptyset \\ 0, & \text{otherwise.} \end{cases} \tag{19}$$

During the online phase, the sub-RSs $\mathcal{R}^i_{\text{off},k}$ from the offline graph are projected onto the current vehicle's initial state p to construct the corresponding online RS $\mathcal{R}^i_{\text{on},k}$. The state transition relation matrix of the online RS is identical to that of the offline RS. These grid-partitioned sub-RSs and their transition relation matrices can be used for high-level modules such as state propagation, trajectory search, and obstacle avoidance optimization, serving as the core supporting structure for the dynamic programming algorithm in this study.

B. Online RS Adjustment Based on Traffic Knowledge

The online RS needs to be adjusted based on the knowledge graph. Using the knowledge graph constructed in

Authorized licensed use limited to: Indian Institute Of Technology Jammu. Downloaded on August 19,2026 at 16:10:56 UTC from IEEE Xplore. Restrictions apply.

Section II, SPARQL queries can be employed to retrieve real-time traffic knowledge $\mathcal{L}$ regarding surrounding vehicles and road infrastructure. Query functions $E_p(\mathcal{L})$, $E_v(\mathcal{L})$, and $E_k(\mathcal{L})$ are introduced to retrieve position, speed, and time constraints from $\mathcal{L}$. Traffic knowledge can be classified into the following four types. For each constraint type, a boolean vector $o_{*,k}$ represents the existence of the sub-RS under traffic knowledge $\mathcal{L}$.

1) *Spatial Restrictions:* This category involves prohibited areas where vehicles are not allowed to travel. Examples include solid road boundary lines, stop lines, and obstacles. These areas are off-limits based on their spatial characteristics

$$o^i_{\mathrm{sr},k} = \begin{cases} 1, & \text{if } \left(Q_p\left(\mathcal{R}^i_{\mathrm{on},k}\right) \cap E_p\left(\mathcal{L}\right)\right) \neq \emptyset \\ 0, & \text{otherwise.} \end{cases} \tag{20}$$

2) *Spatial + Speed Restrictions:* In these areas, vehicles are allowed to pass but must adhere to speed limits. Examples include pedestrian crossings and intersections, where the vehicle can proceed but must control its speed when passing through

$$o^i_{\mathrm{ssr},k} = \begin{cases} 1, & \text{if } \left(\left(Q_p\left(\mathcal{R}^i_{\mathrm{on},k}\right) \cap E_p\left(\mathcal{L}\right)\right) \neq \emptyset\right) \\ & \quad \wedge \left(\left(Q_v\left(\mathcal{R}^i_{\mathrm{on},k}\right) \cap E_v\left(\mathcal{L}\right)\right) \neq \emptyset\right) \\ 0, & \text{otherwise.} \end{cases} \tag{21}$$

3) *Spatial + Temporal Restrictions:* This applies to dynamic traffic situations, such as reversible lanes, or sudden accidents ahead. Vehicles can travel within a specific time window and location, following the traffic rules based on IoT information updates:

$$o^i_{\mathrm{str},k} = \begin{cases} 1, & \text{if } \left(\left(Q_p\left(\mathcal{R}^i_{\mathrm{on},k}\right) \cap E_p\left(\mathcal{L}\right)\right) \neq \emptyset\right) \\ & \quad \wedge \left(Q_t\left(\mathcal{R}^i_{\mathrm{on},k}\right) \in E_t\left(\mathcal{L}\right)\right) \\ 0, & \text{otherwise.} \end{cases} \tag{22}$$

4) *Spatial + Temporal + Speed Restrictions:* These are commonly seen at dynamic traffic intersections. Before entering and while passing through the intersection, vehicles must follow different speed limits based on their position. In addition, changes in traffic light signals communicated through IoT systems further impact the vehicle's planning in such areas:

$$o^i_{\mathrm{stsr},k} = \begin{cases} 1, & \text{if } \left(\left(Q_p\left(\mathcal{R}^i_{\mathrm{on},k}\right) \cap E_p\left(\mathcal{L}\right)\right) \neq \emptyset\right) \\ & \quad \wedge \left(Q_t\left(\mathcal{R}^i_{\mathrm{on},k}\right) \in E_t\left(\mathcal{L}\right)\right) \\ & \quad \wedge \left(\left(Q_v\left(\mathcal{R}^i_{\mathrm{on},k}\right) \cap E_v\left(\mathcal{L}\right)\right) \neq \emptyset\right) \\ 0, & \text{otherwise.} \end{cases} \tag{23}$$

After adjustment based on traffic knowledge, the existence of sub-RSs at time step k is represented using a boolean vector r_k. Based on the aforementioned boolean vectors $o_{*,k}$, r_k is recalculated. The r_k will remain true if and only if: 1) r_k is currently true, and 2) all constraint detection signals $o_{\mathrm{sr},k}, o_{\mathrm{ssr},k}, o_{\mathrm{str},k}, o_{\mathrm{stsr},k}$ are false

$$r_k \wedge \neg\left(o_{\mathrm{sr},k} \vee o_{\mathrm{ssr},k} \vee o_{\mathrm{str},k} \vee o_{\mathrm{stsr},k}\right) \rightarrow r_k. \tag{24}$$

The existence vector can propagate to the next time step r_{k+1}, using the adjacency matrix of the RS. This allows sub-RSs to evolve across time steps based on both dynamic feasibility and traffic knowledge. To provide initial values for the next time step r_{k+1}, r_k propagates by

$$r_{k+1} = T_k^{k+1} r_k. \tag{25}$$

To prevent system failures during operation, a fault detection mechanism is incorporated into the system design. Specifically, when the boolean vector r_k at a given time step k consists entirely of zeros—indicating that the reachable sub-RS is empty—the system is considered to have encountered a failure. Upon detection of such a failure or identification of a critical vehicle state, the system switches to an emergency safety mode, executing controlled braking or transferring control to a fallback mechanism (e.g., human intervention).

In ethical dilemma scenarios (e.g., unavoidable conflicts between objects), the principle of "prioritizing the safety of vulnerable road users" is applied. For sub-RSs occupied by pedestrians or passenger vehicles, these regions are consistently treated as containing obstacles. In contrast, those corresponding to lane markings, construction zones, or similar elements may, depending on situational assessment, be treated as nonobstacles, thereby enabling the vehicle to attempt the replanning of an emergency avoidance trajectory.

IV. RS-Based Path Planning

The process of obtaining the vehicle driving trajectory is illustrated in Fig. 5. It involves selecting the driving corridor, choosing the sub-driving corridor, and then generating the driving trajectory.

A. Extraction of Driving Corridors

At certain time steps, the RS may have disconnected areas on the road due to traffic knowledge. The mth disconnected area at the time step k is denoted as drivable area $\mathcal{A}_{k,m}$. The drivable area $\mathcal{A}_{k,m}$ contains multiple sub-RSs. While different drivable areas in one time step are not connected to each other, the sub-RSs within a single drivable area are interconnected in position. Driving corridors $\mathcal{D}$ are a temporal sequence of connected drivable areas $\{A_{k,m}\}_{k=0}^{K}$ in the position domain. Since there may be multiple selectable driving areas at each time step, multiple driving corridors can be generated. The ith driving corridor is denoted as $\mathcal{D}_i$.

This study introduces a spatial-margin-based corridor evaluation criterion to select the optimal driving corridor. Specifically, the evaluation metric for each candidate corridor is defined as the total area of all its constituent feasible regions $A_{k,m}$: A larger area indicates that the driving corridor is safer and more conducive to obtaining driving trajectories

$$\mathrm{Score}\left(\mathcal{D}_i\right) = \sum_{k=0}^{K} \sum_{m=1}^{n} \mathrm{Area}\left(A_{k,m}\right) \tag{26}$$

where $\mathrm{Area}(A_{k,m})$ represents the 2-D spatial area of region $A_{k,m}$.

Authorized licensed use limited to: Indian Institute Of Technology Jammu. Downloaded on August 19,2026 at 16:10:56 UTC from IEEE Xplore. Restrictions apply.

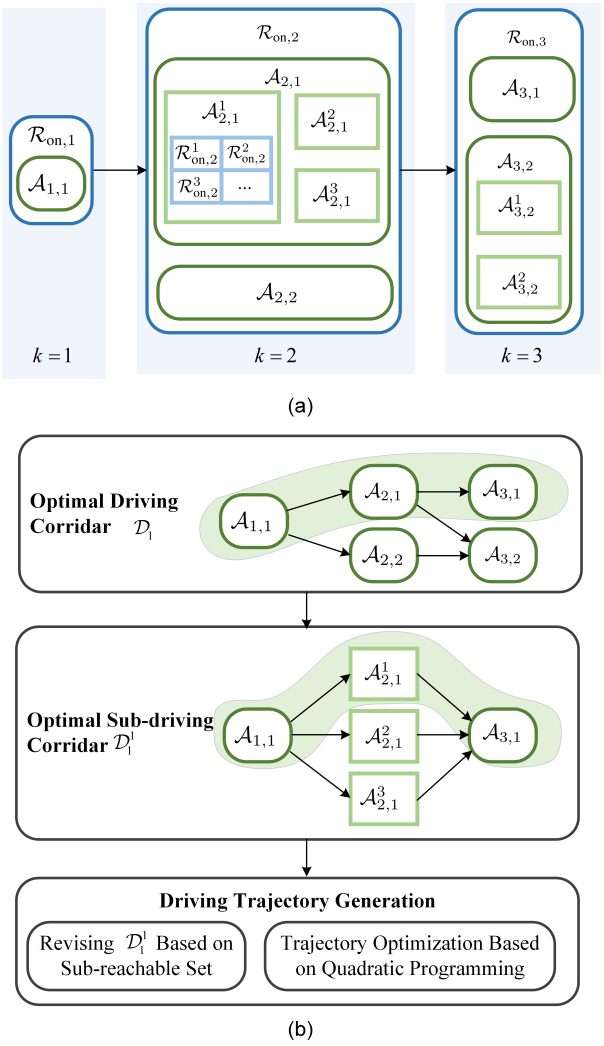


Fig. 5. Trajectory generation based on RSs. (a) Example of drivable areas and driving corridors. (b) Green areas represent the corresponding driving corridor $\mathcal{D}_1$ and sub-driving corridors$\mathcal{D}^l_1$. After extracting the optimal driving corridor and sub-driving corridors, the driving trajectory is generated.

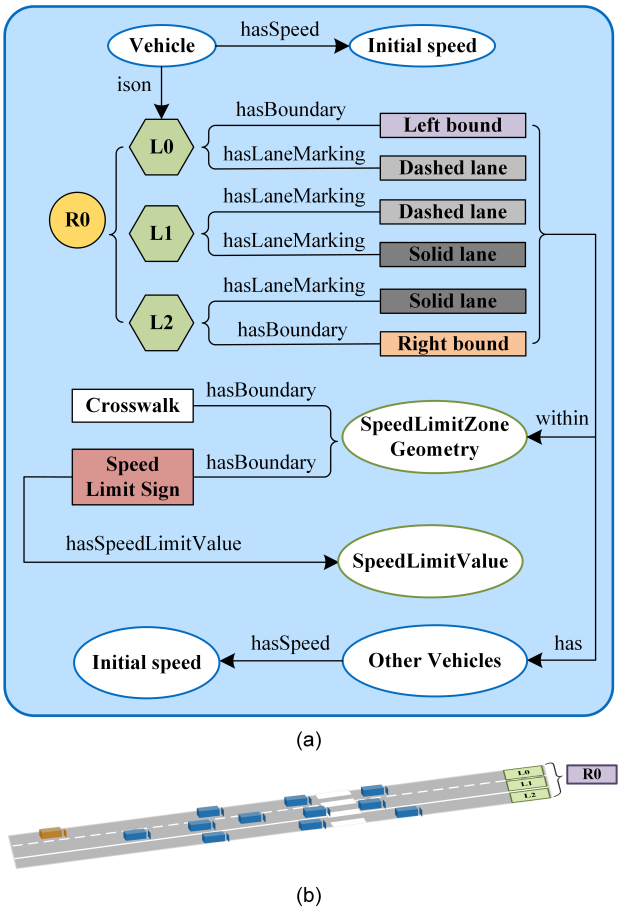


Fig. 6. Static traffic knowledge scenario. (a) Knowledge graph. (b) Road scenario.

B. Extraction of Sub-Driving Corridors

In the optimal driving corridor $\mathcal{D}_o$, some time step covers a large drivable area, leading to significant differences in the vehicle's speed, acceleration, and position. This makes it difficult to generate an optimal trajectory and results in a complex trajectory-solving problem. Therefore, the concept of a "sub-driving corridor" is introduced. A sub-driving corridor $\mathcal{D}^w_o$ is generated by further dividing the drivable area in driving corridor $\mathcal{D}_o$ into smaller sub-drivable areas.

For each sub-RS in the optimal driving corridor, a cost function f_c is calculated. To ensure the vehicle travels along the center of the road, we define a function f_1, which measures the distance between the center point of the sub-drivable area and the road center. To optimize driving efficiency, a function f_2 is defined, calculating the ratio between the longitudinal distance from the furthest to the nearest point of the sub-drivable area and the longitudinal distance from the furthest to the nearest point of the RS at the current time step. To maintain driving stability, we define a function f_3, which calculates the difference between the center point of the sub-drivable area and its before velocity. By calculating the smallest cost of sub-driving areas from the bottom up, the sub-driving corridor chain with the lowest overall cost is obtained

$$f_c\left(\mathcal{R}^n_{k,i}\right) = \alpha_1 f_1\left(\mathcal{R}^n_{k,i}\right) + \alpha_2 f_2\left(\mathcal{R}^n_{k,i}\right) + \alpha_3 f_3\left(\mathcal{R}^n_{k,i}\right) \tag{27}$$

where α_1, α_2, and α_3 are different weighting coefficients used to adjust the contribution of each sub-function to the overall value.

After obtaining the cost f_c of each sub-RS, dynamic programming is used to solve the optimal chain based on a multitree structure. By calculating the minimum cost to reach each sub-RS from the bottom up, the chain with the lowest overall cost is obtained. The recurrence relation can be expressed as

$$f_{\text{dp}}\left(\mathcal{R}^n_{k,i}\right) = \min_{\mathcal{R}^m_{k-1,j}\in\mathcal{R}^*}\left(f_{\text{dp}}\left(\mathcal{R}^m_{k-1,j}\right) + f_c\left(\mathcal{R}^n_{k,i}\right)\right) \tag{28}$$

where $f_dp(\mathcal{R}^n_{k,i})$ is the minimum cost to reach sub-RS $\mathcal{R}^n_{k,i}$ at time step k; $\mathcal{R}^*$ is the set of sub-drivable areas at time step $k-1$ that can connect to $\mathcal{R}^n_{k,i}$.

The sub-RSs for each step are obtained based on the chain. Single sub-RSs with small areas are unfavorable for optimization. Therefore, nearby sub-RSs with an f_{dp} difference smaller than a certain threshold are merged to form sub-drivable areas. Sub-drivable areas $\mathcal{A}^l_{k,m}$ consist of an integer

Authorized licensed use limited to: Indian Institute Of Technology Jammu. Downloaded on August 19,2026 at 16:10:56 UTC from IEEE Xplore. Restrictions apply.

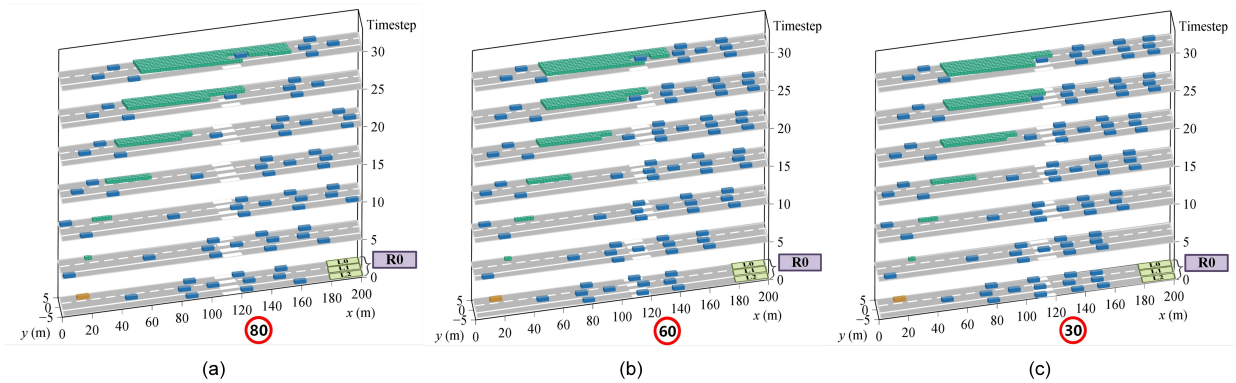


Fig. 7. RSs of the ego vehicle in the static traffic knowledge scenario under the KARS model. (a) Online RSs with the speed limit is 80 km/h. (b) Online RSs with the speed limit is 60 km/h. (c) Online RSs with the speed limit is 30 km/h.

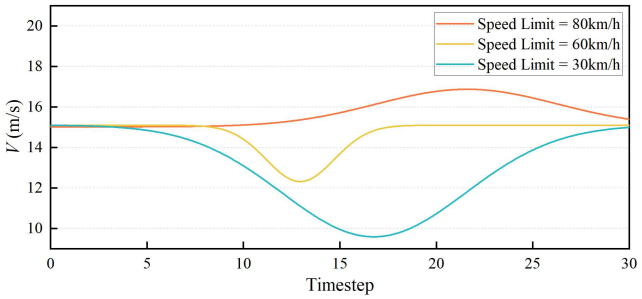


Fig. 8. Velocities of the ego vehicle under the KARS model across three speed limit scenarios.

number of sub-RSs and have a rectangular shape. These sub-drivable areas are connected following the chain of sub-RSs, forming the sub-driving corridor $\mathcal{D}_o^w$.

C. Trajectory Generation

The midpoints p_m of the sub-driving corridor are connected to generate the initial driving trajectory. The optimal trajectory is solved by the sequential quadratic programming method based on the constraints of the sub-driving corridor.

The optimization indices include longitudinal acceleration, longitudinal jerk, and longitudinal distance relative to the optimal node. The cost function $f_q p(x)$ can be expressed as

$$f_q p(x) = \beta_1 U_p^2 + \beta_2 J_p^2 + \beta_2 (p_m - p)^2$$
$$\text{s.t. } p \geq p_{\min}, \quad p \leq p_{\max} \tag{29}$$
$$U_k = \frac{p_k - 2p_{k-1} + p_{k-2}}{\Delta t^2} \tag{30}$$
$$J_k = \frac{p_k - 3p_{k-1} + 3p_{k-2} - x_{p-3}}{\Delta t^3}. \tag{31}$$

In this formulation, $\beta_1, \beta_2, \beta_3$ are the weight coefficients, $p_{\min}, p_{\max}$ define the longitudinal boundaries of the sub-driving corridor, U_k represents the cost of longitudinal acceleration at time step k, U_p is the total acceleration cost accumulated from t_0 to $t_0 + k\Delta t$, J_k is the cost of longitudinal jerk at time step k, J_p is the overall jerk cost accumulated from 0 to $t_0 + k\Delta t$, and p_m is the longitudinal optimal point at time step k.

V. Experiment

In this study, three experiments are conducted. Based on traffic knowledge, road scenarios are divided into static, semi-dynamic, and dynamic scenarios. In static scenarios, traffic knowledge remains unchanged, such as lane markings and speed limits. In semidynamic scenarios, traffic knowledge remains static for a period before changing, such as in construction zones or tidal lanes. In dynamic scenarios, traffic knowledge is constantly changing, such as at traffic light-controlled intersections or on a road where the vehicle in front decelerates suddenly. We construct experimental scenarios based on each type of scenario and its typical traffic knowledge. Specifically, the experiments were conducted on a system with an AMD Ryzen 9 7945HX CPU, NVIDIA RTX 4060 GPU, Ubuntu 24.04 OS, and Python 3.11.

A. Validation of Static Traffic Knowledge Scenario

This study extracts real traffic data from the NGSIM dataset [19] for validation purposes. The NGSIM dataset records various scenarios and includes real road maps, such as intersection scenarios and straight road scenarios. It also records traffic infrastructure, including traffic lights and road signs. Scenarios from the NGSIM dataset are extracted to create a validation dataset for the experiments.

Although NGSIM covers some traffic knowledge, there are still scenarios with more complex traffic constraints that may occur in real life. To our knowledge, there is currently no specialized dataset that focuses on scenarios with complex traffic knowledge. To address this gap, we selected a subset of NGSIM scenarios for further processing, adding more intricate constraints, such as a pedestrian crosswalk. To ensure that surrounding vehicles adhere to these new traffic rules, we implemented the intelligent driver model (IDM) and the MOBIL model [20], [21] for surrounding vehicles, ensuring their response to the newly introduced traffic knowledge.

To validate the feasibility of the KARS model in static traffic knowledge scenarios, we set up a scenario with traffic knowledge illustrated in Fig. 6(b). As shown in Fig. 6(a), a knowledge graph based on the traffic knowledge of the scenario in Fig. 6(b) is created. The scenario includes two traffic constraints. Vehicles are only permitted to cross the

Authorized licensed use limited to: Indian Institute Of Technology Jammu. Downloaded on August 19,2026 at 16:10:56 UTC from IEEE Xplore. Restrictions apply.

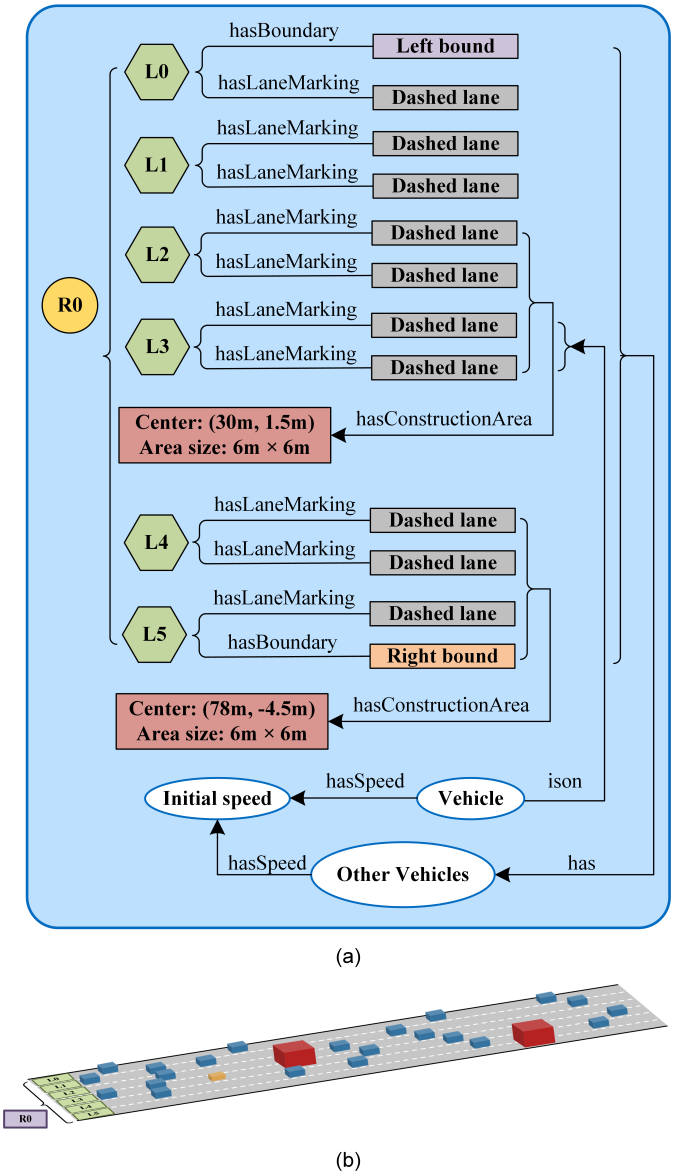


Fig. 9. Semidynamic traffic knowledge scenario. (a) Knowledge graph. (b) Construction areas scenario.

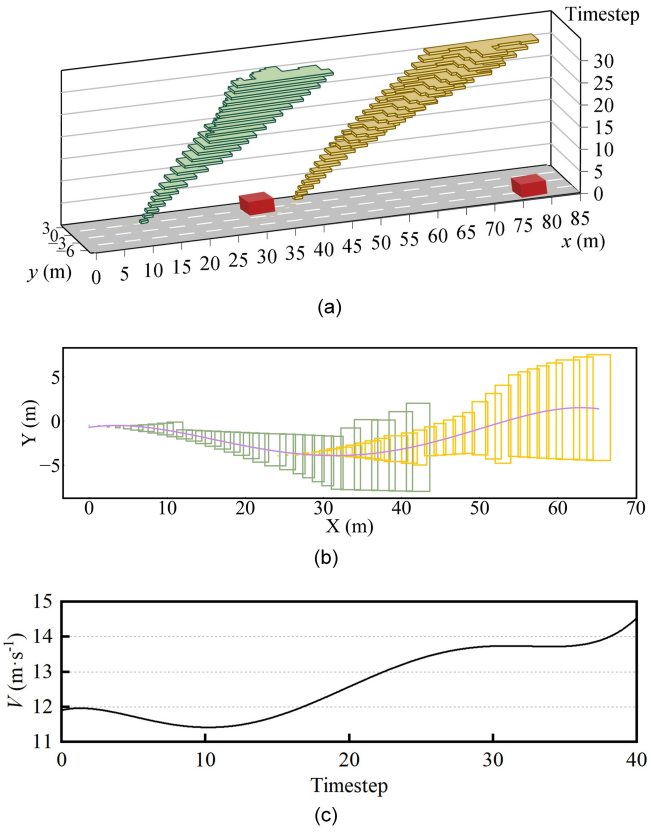


Fig. 10. Planning illustration of the ego vehicle under the KARS model in the semidynamic traffic knowledge scenario. (a) RSs. (b) Driving corridors. (c) Velocity of the ego vehicle.

dashed lane markings, while they are prohibited from crossing solid lane markings. Vehicles are required to slow down under the speed limit values before the crosswalk. In this scenario, the ego vehicle is controlled by the KARS model proposed in this article. In this experiment, the RS is calculated for 30 time steps, with a time interval of 0.1 s. The sub-RS is discretized to squares with a side length of 1m.

The online RS augmented by traffic knowledge is illustrated in Fig. 7. At dashed lane markings, the ego vehicle is allowed to cross the lane line, so the RS retains the relevant sub-RS grids for once right lane change. At solid lane markings, the RS grids exclude sub-RSs that would cross these lines, ensuring that the ego vehicle does not consider twice right lane changes.

The ego vehicle considers the need to decelerate at a pedestrian crosswalk and follows the corresponding road speed limit values, resulting in RS that comply with traffic knowledge. On the pedestrian crosswalk, sub-RSs with high speed are removed. In addition, backward recursion eliminates sub-RSs with high speeds before reaching the pedestrian area, ensuring a smooth deceleration process for the vehicle. Road speed limit values for three scenario in Fig. 7 are $80, 60, 30$ km/h comparatively. The area of the online RS of Fig. 7(c) is the least. The area of the online RS of Fig. 7(b) is less than that of the online RS of Fig. 7(a). The velocity of the ego vehicle at different road speed limits is shown in Fig. 8. When the speed limit is 80 km/h, it is significantly higher than the speed of the ego vehicle, so there is no need to decelerate. At 60 km/h, the ego vehicle chooses to decelerate moderately. At a 30 km/h speed limit, the ego vehicle decelerates below the speed limit.

The sub-RSs obtained from the proposed KARS model provide the ego vehicle with an RS that adheres to traffic knowledge. By removing the sub-RS, the complexity of generating driving corridors is reduced, enabling the generation of optimal speeds.

B. Validation of Semidynamic Traffic Knowledge Scenario

In this experiment, single-step RS planning allows the vehicle to compute for 30 time steps, with a time interval of 0.1 s. In this experiment, we recalculate a single-step RS every 1 s to obtain a re-planned trajectory.

Authorized licensed use limited to: Indian Institute Of Technology Jammu. Downloaded on August 19,2026 at 16:10:56 UTC from IEEE Xplore. Restrictions apply.

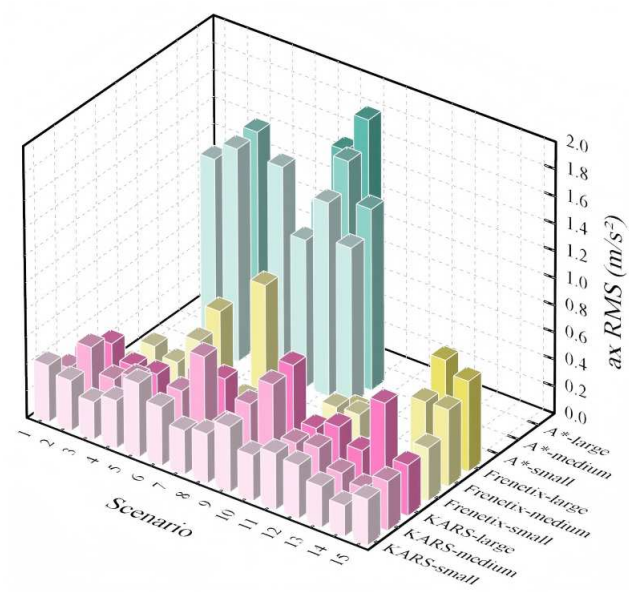


Fig. 11. RMS value of longitudinal acceleration when the algorithm passes through the construction area of different area sizes.

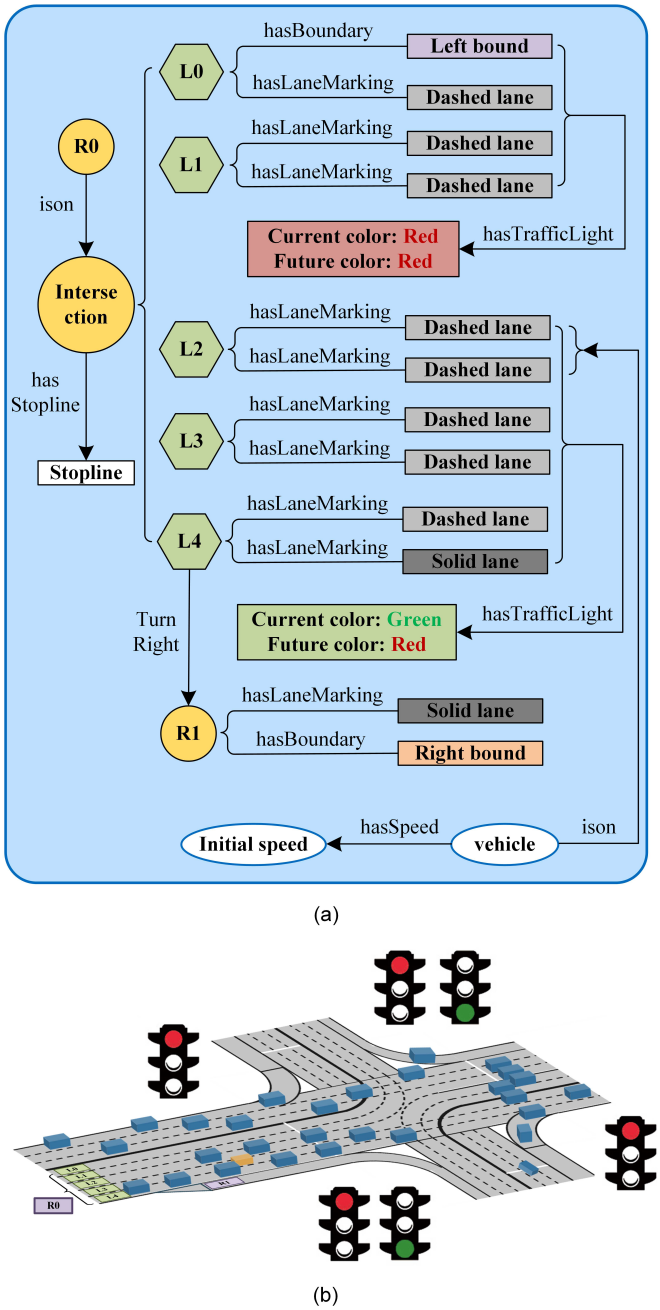


Fig. 12. Dynamic traffic knowledge scenario. (a) Knowledge graph. (b) Traffic light cycle scenario.

This road is a straight lane where dynamic vehicles construction areas exist as shown in Fig. 9(b). The knowledge graph generated based on the traffic knowledge is shown in Fig. 9(a). Construction areas are temporary and their size can vary, making them semi-dynamic traffic knowledge. There are two construction areas, resulting in a small area available for the vehicle to drive. As the vehicle passes the first construction area, it encounters the second construction area, which further hinders its movement. The RS and driving corridor display twice twice-lane-changing trend.

As shown in Fig. 10, the RS and driving corridor exhibit a clear tendency to change lanes to avoid obstacles. Even with limited driving areas, our algorithm still detects a feasible trajectory. Moreover, when generating the RS and driving corridor, it considers potential collisions with surrounding vehicles. The resulting trajectory successfully avoids obstacles with minimal speed reduction, and the selected path is the optimal solution among all available options for obstacle avoidance.

Currently, few planning algorithms are specifically developed to integrate traffic knowledge. For comparison, we select two planning algorithms that can effectively handle specific traffic regulations. The Frenetix model [22] is a sampling-based planning algorithm proposed in 2024, which incorporates lane lines, obstacles, and other factors in feasibility analysis to facilitate planning in high-dynamic environments. What's more, A*, as a widely used graph search algorithm, is well-suited for global path planning and remains a key benchmark in autonomous navigation research.

To evaluate algorithm effectiveness, pass rates in various scenarios are calculated. To assess efficiency, the average velocity of the generated trajectories is computed. For comfort evaluation, the root mean square (rms) of the longitudinal acceleration a_x of the trajectories is measured. Three construction zones of different sizes are designed and placed on the road. The planning performance is assessed using both the proposed method and the comparison algorithm when vehicles encounter construction areas. The larger the obstacle size, the smaller the driving area for the vehicle, making the planning task for the ego vehicle more challenging.

As shown in Table I, when the construction area expands from small to large, the pass rates of both Frenetix and A* plummet (Frenetix drops from 73% to 13%, A* from 40% to 13%), whereas the decline for KARS is much more gradual (100% to 80%), demonstrating its stronger robustness to increasingly constrained spaces. Moreover, across all three sizes, KARS maintains an average velocity above 14.5 m/s,

Authorized licensed use limited to: Indian Institute Of Technology Jammu. Downloaded on August 19,2026 at 16:10:56 UTC from IEEE Xplore. Restrictions apply.

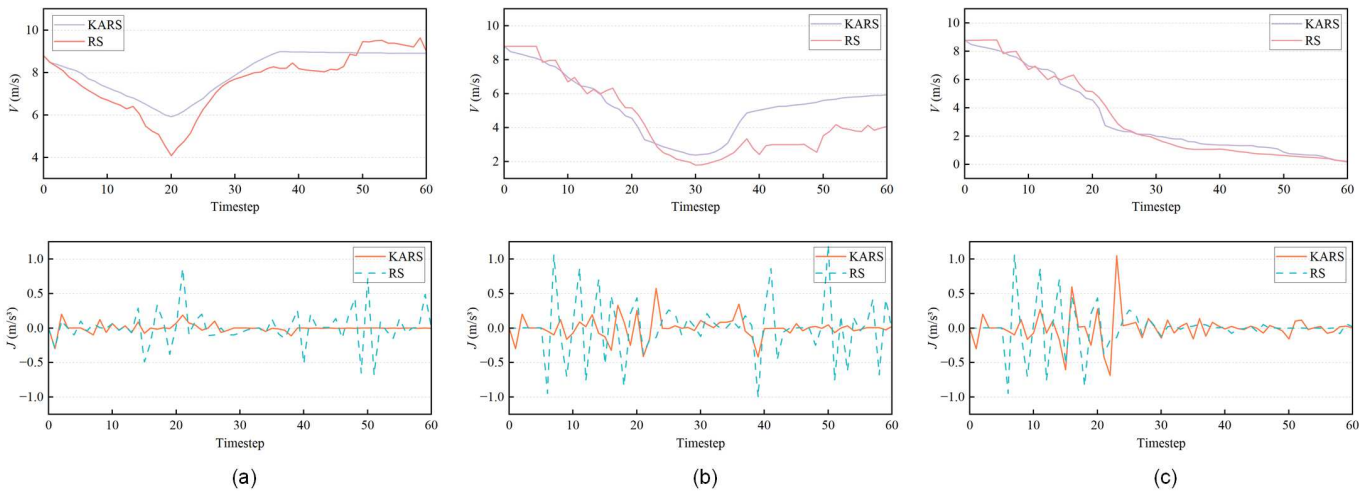


Fig. 13. Illustrations of velocity and jerk of the ego vehicle. (a) Red light turns green after 1 s. (b) Red light turns green after 3 s. (c) Red light turns green after 6 s.

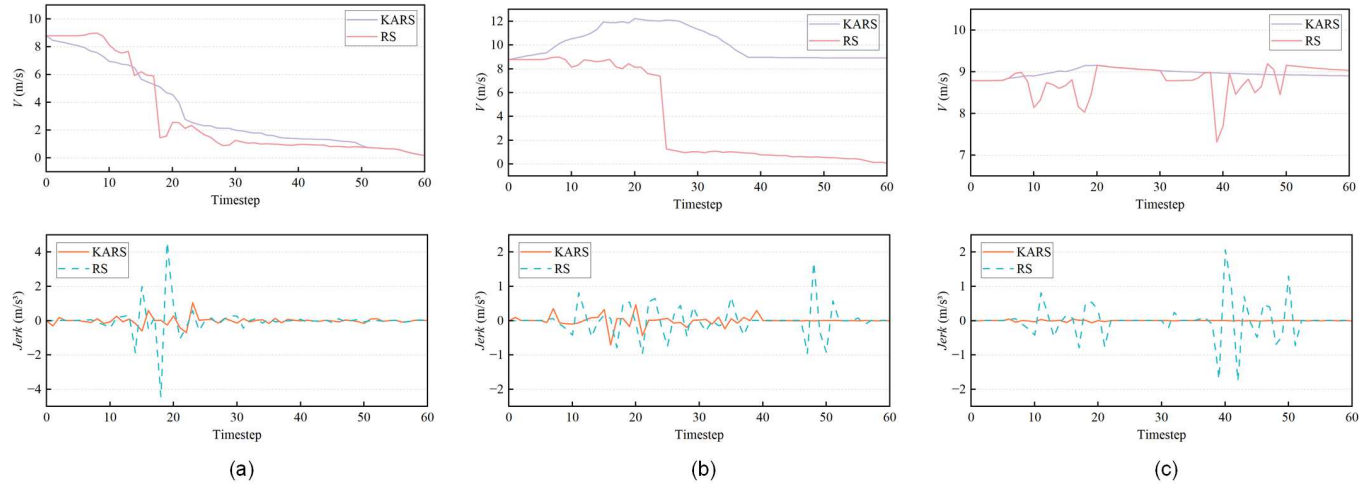


Fig. 14. Illustrations of velocity and jerk of the ego vehicle. (a) Green light turns red after 1 s. (b) Green light turns red after 3 s. (c) Green light turns red after 4 s.

TABLE I
RESULTS IN DIFFERENT CONSTRUCTION AREAS

Construction Area	Method	Pass Rate	Average Velocity (m/s)
Small Size (6m×3m)	KARS	100% 15/15	14.8840
	Frenetix	73.3% 11/15	13.8417
	A*	40.0% 6/15	13.6886
Medium Size (8m×4.5m)	KARS	93.3% 14/15	14.6859
	Frenetix	40% 6/15	13.6140
	A*	33.3% 5/15	13.6908
Large Size (6m×3m)	KARS	80% 12/15	14.5664
	Frenetix	13.3% 2/15	13.1901
	A*	13.3% 2/15	13.6296

whereas the competing algorithms never exceed 14 m/s; even in the large-scale scenario, KARS is still 10.4% faster than Frenetix, highlighting its superior balance between travel efficiency and safety.

As shown in Fig. 11, a_x rms values of the ego vehicle in scenarios are presented. The results indicate that the a_x rms values of KARS are significantly lower than those of Frenetix and A*, suggesting that KARS can handle complex traffic scenarios more smoothly with fewer abrupt accelerations and decelerations. This is mainly attributed to its ability to integrate IoT communication and traffic knowledge, enabling it to predict the positions of obstacles in advance and optimize trajectories. In contrast, Frenetix and A* lack this traffic knowledge integration capability, requiring frequent speed and trajectory adjustments, which lead to higher rms values of longitudinal acceleration and a less smooth driving process.

C. Validation of Dynamic Traffic Knowledge Scenario

In a dynamic traffic knowledge scenario, real-time communication under IoT conditions is allowed. Intersections are dynamic traffic knowledge scenarios with traffic knowledge of the traffic light. The generation of the RS is adjusted in accordance with the traffic light signals. As shown in Fig. 12(b), a single-step RS is planned for 60 time steps. The knowledge graph generated based on the above rules is shown in Fig. 12(a).

Authorized licensed use limited to: Indian Institute Of Technology Jammu. Downloaded on August 19,2026 at 16:10:56 UTC from IEEE Xplore. Restrictions apply.

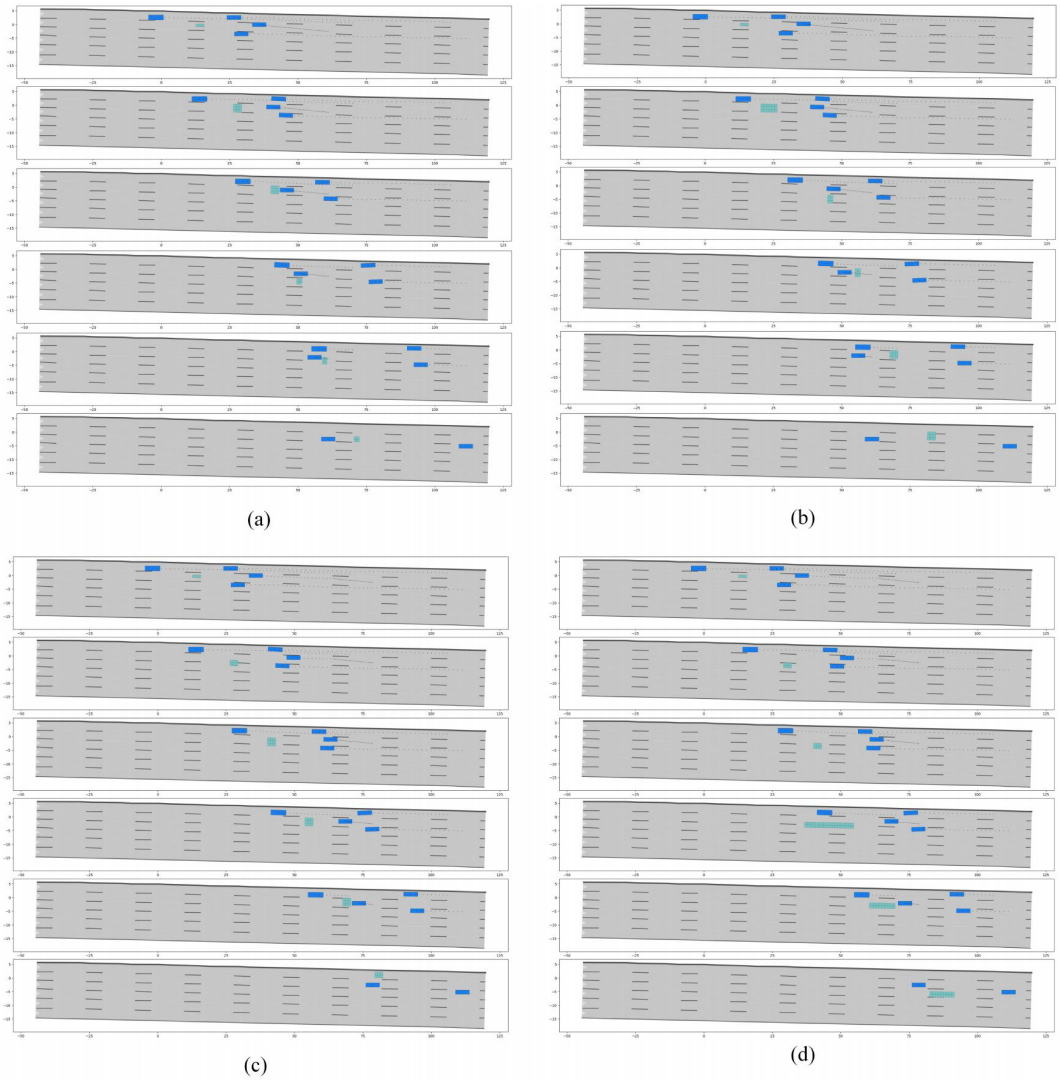


Fig. 15. Comparison of the optimal sub-driving corridor in emergency braking scenarios. (a) and (b) Sub-driving corridor based on the RS model and KARS model in the scenario where the vehicle in front starts to decelerate after 1 s. (c) and (d) Sub-driving corridor based on the RS model and KARS model in the scenario where the vehicle in front starts to decelerate after 3 s.

Different scenarios are analyzed where traffic signals change from red to green and from green to red, with different signal timings designed for each scenario. The RS [18] model, which does not use a knowledge graph, serves as a comparison method.

First, the analysis focuses on the red-to-green signal change. The a_x rms and jerk values of the ego vehicle are calculated for different scenarios where the red light turns green after delays of 1–6 s, as shown in Table II. The changes in speed and jerk for delays of 1, 3, and 6 s are illustrated in Fig. 13.

When the red light turns green after 1 or 2 s, the ego vehicle does not reach the stop line. Using the KARS model, which incorporates traffic knowledge, the ego vehicle anticipates the signal change, resulting in a smaller deceleration. In contrast, the RS model causes the ego vehicle to apply greater deceleration since it cannot predict the red light turning green. For a delay of 3 s, where the ego vehicle is near the stop line, the KARS algorithm enables it to pass through the intersection at a higher speed. However, the RS algorithm results in greater deceleration and a slower acceleration process, ultimately causing the vehicle to pass the intersection at a lower speed. When the red light remains for a longer time or is sustained for

TABLE II
RESULTS IN TRAFFIC LIGHT CHANGING FROM RED TO GREEN

Time	Method	ax RMS (m/s^2)	Max Jerk (m/s^3)
1s	KARS	0.0572	0.2250
	RS	0.1749	0.7823
2s	KARS	0.1313	0.3
	RS	0.2783	0.8621
3s	KARS	0.2368	0.5776
	RS	0.3521	1.2096
4s	KARS	0.2355	0.6015
	RS	0.2978	1.0601
5s	KARS	0.2465	0.5989
	RS	0.2917	1.0601
6s	KARS	0.2432	1.0507
	RS	0.2837	1.0601

Authorized licensed use limited to: Indian Institute Of Technology Jammu. Downloaded on August 19,2026 at 16:10:56 UTC from IEEE Xplore. Restrictions apply.

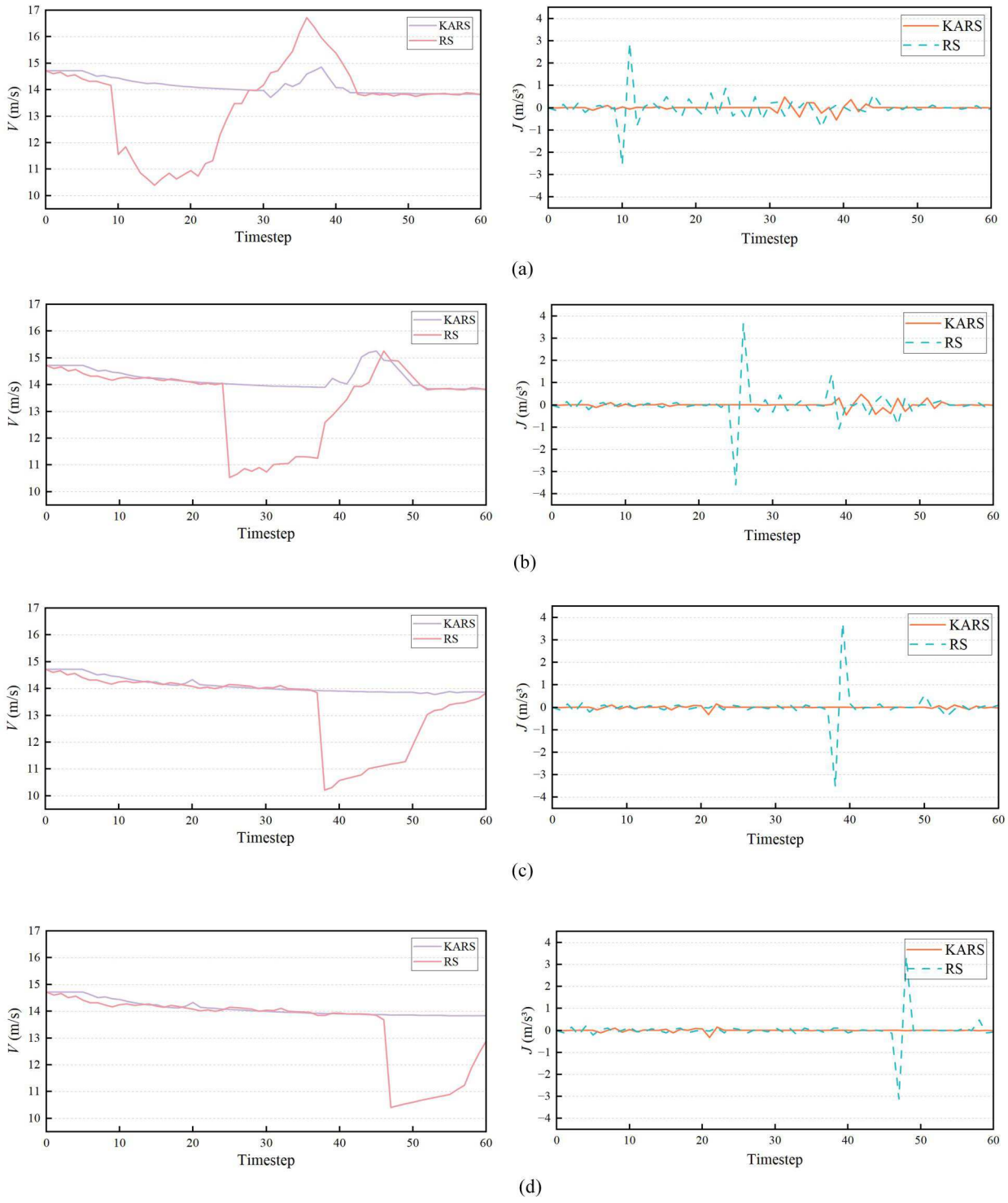


Fig. 16. Velocities of the ego vehicle under the KARS model across three speed limit scenarios. (a) Vehicle in front suddenly decelerates 1 s later. (b) Vehicle in front suddenly decelerates 2 s later. (c) Vehicle in front suddenly decelerates 3 s later. (d) Vehicle in front suddenly decelerates 4 s later.

an extended period, the vehicle reaches the stop line, briefly stops, and then accelerates once the signal turns green.

Next, the analysis examines the green-to-red signal change. The a_x rms and jerk values of the ego vehicle are calculated for different scenarios where the green light turns red after delays of 1 to 4 s, as shown in Table III. The changes in speed and jerk for delays of 1, 3, and 4 s are illustrated in Fig. 14.

When the green light turns red after 1 or 2 s, the KARS model knows the signal change through IoT and the traffic knowledge graph, allowing the ego vehicle to begin decelerating while driving. In contrast, the RS model cannot anticipate the signal change, causing the ego vehicle to maintain a constant speed for 1 or 2 s during the green light, followed by a sharp brake after the light turns red, eventually stopping in

Authorized licensed use limited to: Indian Institute Of Technology Jammu. Downloaded on August 19,2026 at 16:10:56 UTC from IEEE Xplore. Restrictions apply.

TABLE III
RESULTS IN TRAFFIC LIGHT CHANGING FROM GREEN TO RED

Time	Method	ax RMS (m/s^2)	Max Jerk (m/s^3)
1s	KARS	0.2432	1.0507
	RS	0.6476	4.5608
2s	KARS	0.2923	0.9916
	RS	0.6845	4.5025
3s	KARS	0.1860	0.7013
	RS	0.2870	1.6890
4s	KARS	0.0171	0.0548
	RS	0.3644	2.0662

TABLE IV
RESULTS IN THE FRONT VEHICLE'S SUDDEN DECELERATION

Time	Method	ax RMS (m/s^2)	Max Jerk (m/s^3)
1s	KARS	0.1120	0.5496
	RS	0.4546	2.8970
2s	KARS	0.1316	0.4835
	RS	0.5159	3.6551
3s	KARS	0.0446	0.3241
	RS	0.4910	3.7285
4s	KARS	0.0421	0.3242
	RS	0.4424	3.3410

front of the stop line. For a delay of 3 s, the RS model does not pass through the intersection before the light change, resulting in an emergency stop before the intersection. The KARS model, benefiting from precise traffic knowledge, accelerates the vehicle in advance to pass the intersection and then reduces its speed after passing the intersection. In scenarios where the green light remains for a long period, both algorithms allow the vehicle to pass through the intersection smoothly. However, the RS model exhibits slight speed oscillations, which are more pronounced compared to the KARS algorithm.

The road where the vehicle in front brakes suddenly also contains dynamic traffic knowledge. To verify the performance of the KARS model under extreme working conditions, we constructed a set of emergency braking tests where the ego vehicle follows a lead vehicle that suddenly decelerates. These tests are designed to assess whether the planning system can respond effectively to sudden risk. We designed four variations: the lead vehicle begins to brake after 1, 2, 3, and 4 s of forward motion, respectively. We compared the KARS model with the RS model mentioned in the previous text.

As shown in Fig. 15(a) and (b), when using the RS model, the ego vehicle cannot predict risks in advance. Therefore, the ego vehicle only changes lanes when the distance to the vehicle in front is close. But the KARS model can predict risks in advance, allowing for lane changes in advance. In Fig. 15(c) and (d), when planning based on the RS model, due to the sudden deceleration of the vehicle in front, the ego vehicle brakes at a position closer to the vehicle in front. However, with the KARS model, by changing lanes in advance during planning, there is a larger driving area, thus avoiding sudden braking.

As shown in Fig. 16 and Table IV, our model achieves lower rms acceleration and peak jerk, maintaining more stable longitudinal speed. This demonstrates the KARS model's enhanced risk awareness and robustness in handling sudden, unpredictable traffic behaviors. Furthermore, despite this article not directly addressing multiagent cooperation, the model still shows significant improvements.

In summary, with the support of IoT and traffic knowledge in dynamic scenarios, the ego vehicle can adopt more reasonable driving strategies. In response to real-time changes, it can plan rational speeds and trajectories, thereby enhancing driving efficiency and comfort.

VI. CONCLUSION

This article proposed a traffic knowledge-augmented path planning model (KARS), which integrates knowledge graphs with RSs in IoT environments. Traffic knowledge was systematically represented through a knowledge graph in an IoT environment, providing a comprehensive framework for incorporating regulations and dynamic road conditions. By leveraging this information, the RS-based planning model dynamically adapts to traffic knowledge, enabling real-time path planning across a wide range of scenarios.

This framework is built to tackle the key problems in existing methods, specifically fixing the issue where traditional path planning does not fully use traffic-related knowledge. At the same time, it focuses on how traffic knowledge can improve local path planning in the context of IoT.

The feasibility of KARS was validated through experiments conducted in static, semidynamic, and dynamic traffic knowledge scenarios. The results demonstrated that KARS effectively merged diverse traffic knowledge with the planning algorithm, producing trajectories that consistently complied with traffic regulations and road conditions. In complex and dynamic environments, the algorithm significantly improves the traffic success rate, providing reliable assurance for driving safety. Meanwhile, the maximum driving speed of the vehicle has been effectively increased, directly enhancing the overall driving efficiency. In addition, the driving trajectories generated by the algorithm feature better acceleration control and smooth speed change characteristics, fully ensuring comfort during the driving process.

REFERENCES

[1] A. Best, S. Narang, D. Barber, and D. Manocha, "AutonoVi: Autonomous vehicle planning with dynamic maneuvers and traffic constraints," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Sep. 2017, pp. 2629–2636.

[2] X. Lu, H. Zhao, B. Gao, W. Chen, and H. Chen, "Decision-making method of autonomous vehicles in urban environments considering traffic laws," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 21641–21652, Nov. 2022.

[3] J. Han, J. Zhao, B. Zhu, and D. Song, "Spatial–temporal risk field for intelligent connected vehicle in dynamic traffic and application in trajectory planning," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 3, pp. 2963–2975, Mar. 2023.

[4] L.-L. Wang, J.-S. Gui, X.-H. Deng, F. Zeng, and Z.-F. Kuang, "Routing algorithm based on vehicle position analysis for Internet of Vehicles," *IEEE Internet Things J.*, vol. 7, no. 12, pp. 11701–11712, Dec. 2020.

[5] W. Duan, J. Gu, M. Wen, G. Zhang, Y. Ji, and S. Mumtaz, "Emerging technologies for 5G-IoV networks: Applications, trends and opportunities," *IEEE Netw.*, vol. 34, no. 5, pp. 283–289, Sep. 2020.

Authorized licensed use limited to: Indian Institute Of Technology Jammu. Downloaded on August 19,2026 at 16:10:56 UTC from IEEE Xplore. Restrictions apply.

[6] I. Ud Din, A. Almogren, and J. J. P. C. Rodrigues, "AIoT integration in autonomous vehicles: Enhancing road cooperation and traffic management," *IEEE Internet Things J.*, vol. 11, no. 22, pp. 35942–35949, Nov. 2024.
[7] L. Zhang, X. Zhu, and J. Ma, "IoT route planning based on spatiotemporal interactive attention neural network," *IEEE Internet Things J.*, vol. 11, no. 5, pp. 7697–7709, Mar. 2024.
[8] C. Xie, B. Yu, Z. Zeng, Y. Yang, and Q. Liu, "Multilayer Internet-of-Things middleware based on knowledge graph," *IEEE Internet Things J.*, vol. 8, no. 4, pp. 2635–2648, Feb. 2021.
[9] H. Kumar Shakya et al., "Internet of Things-based intelligent ontology model for safety purpose using wireless networks," *Wireless Commun. Mobile Comput.*, vol. 2022, no. 1, Jan. 2022, Art. no. 1342966.
[10] L. Halilaj, J. Luettin, C. Henson, and S. Monka, "Knowledge graphs for automated driving," in *Proc. IEEE 5th Int. Conf. Artif. Intell. Knowl. Eng. (AIKE)*, Sep. 2022, pp. 98–105.
[11] J. Tan, Q. Qiu, W. Guo, and T. Li, "Research on the construction of a knowledge graph and knowledge reasoning model in the field of urban traffic," *Sustainability*, vol. 13, no. 6, p. 3191, Mar. 2021.
[12] T. Sun, J. Xu, and C. Hu, "An efficient algorithm of star subgraph queries on urban traffic knowledge graph," *Data Sci. Eng.*, vol. 7, no. 4, pp. 383–401, Dec. 2022.
[13] L. Mlodzian et al., "NuScenes knowledge graph—A comprehensive semantic representation of traffic scenes for trajectory prediction," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshops (ICCVW)*, Oct. 2023, pp. 42–52.
[14] L. Halilaj, I. Dindorkar, J. Luettin, and S. Rothermel, "A knowledge graph-based approach for situation comprehension in driving scenarios," in *Proc. Semantic Web*, 2020, pp. 699–716.
[15] R. Wickramarachchi, C. Henson, and A. Sheth, "Knowledge graphs of driving scenes to empower the emerging capabilities of neurosymbolic AI," *IEEE Internet Comput.*, vol. 28, no. 6, pp. 62–67, Nov. 2024.
[16] S. Manzinger, C. Pek, and M. Althoff, "Using reachable sets for trajectory planning of automated vehicles," *IEEE Trans. Intell. Vehicles*, vol. 6, no. 2, pp. 232–248, Jun. 2021.
[17] X. Zhang, B. Yang, X. Pei, and S. Lu, "Trajectory planning based on spatio-temporal reachable set considering dynamic probabilistic risk," *Eng. Appl. Artif. Intell.*, vol. 123, Aug. 2023, Art. no. 106291.
[18] M. Klischat and M. Althoff, "A multi-step approach to accelerate the computation of reachable sets for road vehicles," in *Proc. IEEE 23rd Int. Conf. Intell. Transp. Syst. (ITSC)*, Sep. 2020, pp. 1–7.
[19] V. Alexiadis et al., "The next generation simulation program," *Inst. Transp. Eng. J.*, vol. 74, no. 8, pp. 22–26, 2004.
[20] A. Kesting, M. Treiber, and D. Helbing, "General lane-changing model MOBIL for car-following models," *Transp. Res. Rec., J. Transp. Res. Board*, vol. 1999, no. 1, pp. 86–94, Jan. 2007.
[21] M. Treiber, A. Hennecke, and D. Helbing, "Congested traffic states in empirical observations and microscopic simulations," *Phys. Rev. E, Stat. Phys. Plasmas Fluids Relat. Interdiscip. Top.*, vol. 62, no. 2, pp. 1805–1824, Aug. 2000.
[22] R. Trauth, K. Moller, G. Würsching, and J. Betz, "FRENETIX: A high-performance and modular motion planning framework for autonomous driving," *IEEE Access*, vol. 12, pp. 127426–127439, 2024.



Danni Chen received the B.S. degree in vehicle engineering and the M.S. degree in mechanical engineering from Beijing Institute of Technology, Beijing, China, in 2022 and 2025, respectively. She is currently pursuing the Ph.D. degree with the Department of Aeronautical and Aviation Engineering, The Hong Kong Polytechnic University, Hong Kong.

Her research interests include driver behavior modeling, intelligent transportation, and vehicular systems.



Chao Lu received the B.S. degree in transport engineering from Beijing Institute of Technology, Beijing, China, in 2009, and the Ph.D. degree in transport studies from the University of Leeds, Leeds, U.K., in 2015.

In 2017, he was a Visiting Researcher with the Advanced Vehicle Engineering Centre, Cranfield University, Cranfield, U.K. He is currently an Associate Professor with the School of Mechanical Engineering, Beijing Institute of Technology. His research interests include intelligent transportation and vehicular systems, driver behavior modeling, reinforcement learning, transfer learning, and their applications.



Shijie Wu received the B.S. degree in vehicle engineering from Beijing Forestry University, Beijing, China, in 2024. He is currently pursuing the master's degree with the School of Mechanical Engineering and Vehicle Engineering, Beijing Institute of Technology, Beijing.

His research interests include decision-making and trajectory planning of intelligent vehicles.



Xuemei Chen received the B.E. degree in automobile operation engineering from Shandong University of Technology, Zibo, China, in 2000, the M.S. degree from Beijing University of Technology, Beijing, China, in 2003, and the Ph.D. degree from Beijing Institute of Technology (BIT), Beijing, in 2006.

She is currently an Associate Professor with the Mechanical Engineering Department, BIT, and the Executive Vice President of the Advanced Technology Research Institute, BIT, since 2021. Her research interests include driver behavior models, autonomous vehicle decision-making, and machine learning.



Jianwei Gong (Member, IEEE) received the B.S. degree in mechanical engineering from the National University of Defense Technology, Changsha, China, in 1992, and the Ph.D. degree in mechanical engineering from Beijing Institute of Technology (BIT), Beijing, China, in 2002.

From 2011 to 2012, he was a Visiting Professor with the Department of Mechanical Engineering, Massachusetts Institute of Technology, Cambridge, MA, USA. He is currently a Professor and the Director of the Research Center of Intelligent Vehicles, BIT. His research interests include intelligent vehicles, environmental perception, motion planning, and control for autonomous driving.

Authorized licensed use limited to: Indian Institute Of Technology Jammu. Downloaded on August 19,2026 at 16:10:56 UTC from IEEE Xplore. Restrictions apply.